

Deep reinforcement learning based scheduling within production plan in semiconductor fabrication

Young Hoon Lee, Seunghoon Lee^{*,1}

Department of Industrial Engineering, Yonsei University, 50 Yonsei-ro Seodaemun-gu, Seoul 03722, Republic of Korea

ARTICLE INFO

Keywords:

Deep reinforcement learning
Production planning
Scheduling
Semiconductor fabrication
Intelligent manufacturing

ABSTRACT

In the semiconductor industry, efficient production planning and scheduling decisions are required to enhance the manufacturing productivity of a company as the system is complicated due to a re-entry characteristic and requires a long production lead time. Production planning is implemented before scheduling and is important for successful manufacturing operations. However, if scheduling at the operation level cannot execute the production plan, failures occur because of inconsistent decisions. Therefore, scheduling needs to fulfill the production plan to ensure realistic decision-making processes for the companies aiming for economic growth and global competitiveness. In this study, deep reinforcement learning (RL) is employed to deal with a scheduling process operating within the production plan. As the algorithm of the deep RL, Deep Q-network is conjugated, and a novel state, action, and reward are suggested to optimize the scheduling policy. As a result, the performance of the proposed deep RL method is in comparison with other dispatching rules, and the proposed method outperforms the other scheduling methods in diverse cases.

1. Introduction

The arrival of Industry 4.0 in manufacturing enables advanced technologies such as the Internet of Things (IoT), cloud computing, and big data to communicate and cooperate with machines for decision-making in real-time by monitoring physical processes (Tao et al., 2019; Zhong et al., 2017). This introduces a new manufacturing paradigm, namely smart manufacturing (Wang et al., 2018). Smart manufacturing is defined as the creation of intelligence based on simulation and optimization through data transmission and analysis in real-time to positively impact all aspects of manufacturing (Tao et al., 2018). With Industry 4.0 and smart manufacturing, the production system has become more intelligent and flexible and is positioned as a key system for improving the productivity and efficiency of manufacturing companies.

In the production system, production planning and scheduling are core processes of manufacturing companies to maximize their profit as the processes are directly related to productivity and efficiency. Production planning is carried out in the strategic decision-making phase and determines the production quantity of products and parts daily, weekly, or monthly considering the status of manufacturing lines,

customer demand, and so forth. Scheduling is executed in the operational decision-making phase and its main goal is to fulfill the production plans. To achieve the goal, scheduling runs every second or minute and assigns jobs to machines. In both production planning and scheduling, there exist dynamic and uncertain factors. For example, in production planning, there are uncertainties such as customer demand and raw material supply. Unexpected machines' breakdown and new jobs' insertion, etc. occur in the scheduling. Scheduling is in a more dynamic environment than production planning in the perspective of performing at a higher frequency.

In semiconductor manufacturing, the fabrication of a wafer involves several hundred process steps and machines. In addition, the re-entry characteristic complicates the semiconductor fabrication system. Unlike other manufacturing industries, the manufacturing period is much longer, which may increase the uncertainty in manufacturing. Therefore, robust scheduling is required to achieve the target production plan in a dynamic environment.

Scheduling has been studied by several researchers and practitioners, and the methodology for scheduling is largely divided into three categories: optimization, heuristic, and dispatching rules. While optimization techniques can find an optimal value or a value close to the optimal

* Corresponding author.

E-mail address: shbrandonlee@yonsei.ac.kr (S. Lee).

¹ ORCID: <https://orcid.org/0000-0003-2538-4074>.

solution, using them is restricted in dynamic environments because the data cannot be reflected in real-time. In addition, an optimal solution may not be able to be found within a reasonable time when the size of a problem is large.

Heuristic methods complement such problems of the optimization techniques. The advantage of heuristic methods is that a good solution can be found rapidly, but the disadvantage is the lack of flexibility. For instance, when a new machine is added to a production line, the heuristic may be reconfigured to provide a valid solution. In the case of dispatching rules, although they can find a reasonably good solution within a relatively short time and are simple, a poor solution can be found unexpectedly. These methodologies have limited application in a dynamic environment and are unable to address the uncertainty owing to the lack of intelligence and flexibility. Recent efforts led to the proposal of deep reinforcement learning (RL), which is remarkably capable of addressing complex problems. Deep RL is a combination of reinforcement learning and deep learning, which is a neural network (NN). RL consists of an agent and environment, and they interact with each other. The agent observes the state of the environment and takes action from a set of actions. Thereafter, the environment rewards the agent. In a sequential decision-making process, the NN is utilized to select the action. By repeating this process, the goal of RL is to teach the agent, which can be rewarded as much as possible in a given environment.

Several studies have solved scheduling problems in semiconductor manufacturing with deep RL, which enables a decentralized, self-learning, and self-optimizing system and provides flexibility and global optimization (Waschneck et al., 2018b). However, to date, research focused on the scheduling itself, such as minimizing completion time and maximizing throughput, rather than considering the production plan.

In this study, a scheduling method is proposed using deep RL for fulfilling the production plan generated from a production planning model. Novel formulations of the state, action, and reward are proposed to accommodate the production plan and variability of the production environment. In addition, a centralized NN is employed to embrace the change in the number of machines. A deep Q-network (DQN) is employed as the deep RL algorithm, and the performance of deep RL is compared with other scheduling methods, namely dispatching rules and a deep RL method presented by others.

The remainder of this paper is organized as follows. Section 2 presents a literature review on scheduling in the semiconductor industry and deep RL. In Section 3, the details of the semiconductor fabrication line that focuses on wafer fabrication are described. In Section 4, the production-planning model is discussed, and the deep RL algorithm with the Markov decision process is explained. In Section 5, details of experiments with the deep RL algorithm are presented in comparison with other scheduling methods. Lastly, conclusions are stated in Section 6.

2. Literature review

Many studies have addressed scheduling in semiconductor fabrication, as it is a complex system, and assessed the performance of the proposed methods. In the past, mathematical programming, heuristics, and dispatching rules have been addressed mainly for semiconductor fabrication. Although artificial intelligence (AI) has been discussed as well, it was fewer studies than the other methods and has not been addressed actively. Before the era of Industry 4.0, the following studies have been conducted chiefly.

Mathematical programming and operational research (OR) methodologies are employed to find the global and deterministic optimal solutions (Zhang et al., 2019). Branch and Bound (B&B) and Lagrangian relaxation (LR) are popular techniques with which to solve integer-programming problems. Sung and Choung (2000) applied B&B to a single burn-in oven, a batch processing machine, in semiconductor fabrication, and the proposed B&B was designed to minimize the makespan of all jobs. Some authors used the LR technique to solve

scheduling problems. Liao et al. (1996) modeled a research and development (R&D) pilot line for semiconductor wafer fabrication and applied the LR to machine capacity constraints. Kaskavelis and Carmanis (1998) proposed the effective redesign of the Lagrange multiplier update applied to scheduling problems with over 10,000 resource constraints. Other methods for mathematical programming include decomposition and queuing network models

(Connors et al., 1994; Demirkol & Uzsoy, 1997; Kumar & Kumar, 2001; Sourirajan & Uzsoy, 2007).

Dispatching rules, which have been widely used for scheduling in semiconductor fabrication, constitute a method to efficiently obtain a good solution to complicated scheduling problems. Wein (1988) discussed the impact of scheduling on the performance of semiconductor fabrication. Twelve dispatching rules were compared in three different fab models. Lu et al. (1994) proposed fluctuation smoothing policies that showed the best mean cycle time and standard deviation for the cycle time in the three fab models employed by Wein (1988). Kim, Lee, Kim, and Roh (1998) considered lot release control, mask scheduling, and batch scheduling simultaneously. The proposed rules outperformed existing rules in performance measurements. Tsai et al. (2003) considered a normal condition, low work in process (WIP) level in bottleneck machines, high WIP levels in non-bottleneck machines, and batch machines in their dispatching rules to minimize the WIP level and cycle time as well as to protect the output. Chen * et al. (2004) developed a dynamic dispatching rule based on the state. For example, workstations were grouped according to a dynamic bottleneck and a non-dynamic bottleneck. Then, the rule of two boundaries (TB) and the shortest expected processing time until its next visit for the long and short queues, respectively, were applied to the dynamic bottleneck. In the non-dynamic bottleneck, the FGCA rule (the definition thereof was not provided by Wein (1988)) was employed. From a daily perspective, scheduling has been developed for semiconductor manufacturing (Lee & Kim, 2011). Standards for dispatching rules were combined into a single rule (Dabbas & Fowler, 2003).

Other researchers employed meta-heuristic methods for scheduling in semiconductor fabrication. Wen et al. (2001) presented a genetic algorithm-based scheduler that addressed dispatching rules, setup rules, and batching rules as the chromosome. With a schedule builder, the chromosome was transformed into a feasible solution. In comparison with other dispatching rules, the GA delivered superior performance. Similarly, another GA was presented with the chromosome expressed using the machine select rule, batch rule, and dispatching rule and showed higher movement, throughput, and shorter mean cycle time than the two dispatching rules used in the study (Qiao et al., 2013). Jain et al. (2003) employed simulated annealing that considered dispatching, setup, and a batching rule to minimize the mean cycle time and tardiness and was shown to outperform other existing rules in terms of the two objectives. Li (2012) proposed a multi-ACO method to find the scheduling solution for bottlenecks. Another study (Guo et al., 2012) presented decomposition-based classified ACO, in which a large scheduling problem was decomposed into several problems.

However, as the era of Industry 4.0 has come, the manufacturing environment has changed with advanced technologies and computing power. With cloud computing, the IoT, deep learning, and so forth, the production efficiency increased, and the manufacturing environment became more advanced. The AI methods for scheduling have been studied and discussed by researchers and practitioners to utilize for better production efficiency.

In the AI methods, recently, deep reinforcement learning (RL) has emerged as a new way to enhance the efficiency of scheduling. The combination of deep learning and RL, known as deep RL, performs more powerfully than before, and scheduling problems using deep RL have been addressed in diverse fields.

Scheduling with deep RL has been studied for use in semiconductor manufacturing as well. Some authors focused on a scheduling problem for minimizing the makespan using the deep RL. Waschneck et al.

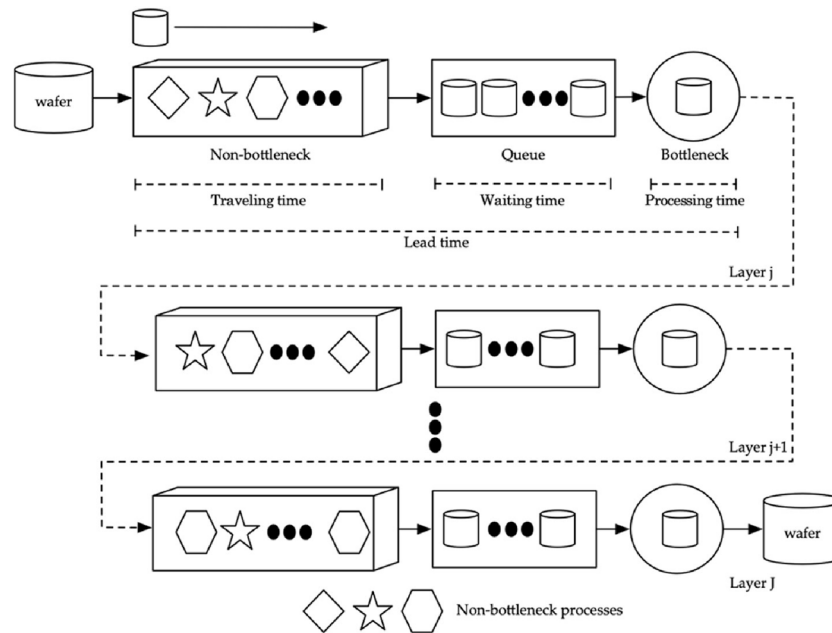


Fig. 1. Wafer fabrication process.

(2018a) designed deep RL to minimize the cycle time for semiconductor production scheduling. Park et al. (2019) paid attention to multichip products' scheduling in semiconductor manufacturing and compared other scheduling methods for minimizing the makespan. Other authors addressed scheduling problems by the deep RL regarding utilization and processing time ratio. Stricker et al. (2018) proposed deep RL for adaptive order dispatching in semiconductor fabrication and showed higher performance in terms of system utilization. Shi et al. (2020) tested the presented deep RL for three different production line environments and achieved competitive scheduling by comparing heuristic scheduling in terms of the processing time ratio on a crucial path. Luo (2020) addressed the new job insertions to minimize the total tardiness in the dynamic job shop scheduling problem, semiconductor manufacturing process, by the deep RL and showed better results than other rules.

A number of studies have focused on scheduling using deep RL for maximizing the throughput, utilization, and minimizing the completion time, and so forth. As mentioned above, as a long production period is required for semiconductor fabrication, fulfilling the production plan is important to stabilize the production line in a dynamic manufacturing environment. In this study, deep RL is employed to provide robust and intelligent scheduling in a dynamic environment to execute a production plan.

3. Semiconductor fabrication description

In this study, the assumptions and processes of the semiconductor fabrication line developed by Kim and Lee (2016) were employed. The manufacturing semiconductor process largely comprises wafer fabrication, wafer probe, assembly (packaging), and final testing. This study focuses on wafer fabrication, which is the most complex process and has a long production time. Wafers are moved in batches, and each wafer visits the same machine several times. A wafer consists of several layers that require typical operations such as cleaning, oxidation, deposition, metallization, lithography, etching, ion implantation, photoresist stripping, inspection, and measurement. Each wafer passes through the operation several times, which entails a considerable amount of recirculation (Pinedo, 2012).

In this study, the fabrication line is modeled on the basis of photolithography, which is the main bottleneck in the fabrication process.

This situation arises because photolithography (hereafter abbreviated as photo) machines are highly expensive in comparison with other machines, and hence fewer machines are available; thus, buffering can easily occur in the photo process.

Except for this process, other processes such as etching and oxidation are regarded as non-bottleneck processes. As mentioned, the layers of the wafer need to be subjected to various operations, and the lead-time for the operations consists of the summation of the traveling time, waiting time, and processing time. The traveling time refers to the moving time in the non-bottleneck process, and the waiting time is the time the wafers are in the queue waiting to proceed to the bottleneck process. Finally, the processing time is the time required for processing on a machine in the bottleneck process. Fig. 1 illustrates the overall wafer fabrication process from the non-bottleneck to the bottleneck process.

As shown in Fig. 1, the non-bottleneck process is expressed as a single process. In the process, the capacity and setup change are not taken into consideration. On the other hand, the setup change and capacity of the machines in the bottleneck process are considered. As wafer fabrication is a flexible manufacturing system, each layer of each type of wafer is deposited using several dedicated machines in the bottleneck process and can be processed on one of them. Wafers are required to wait in the queue until the dedicated machines for wafers are idle if they are busy in the bottleneck process. Once they become idle, the wafers in the queue are allocated to them to be processed. When the type of wafer differs from that of the previous wafer on the same machine, a setup change occurs, which takes additional time in addition to the processing time.

4. Production planning and scheduling modeling

4.1. Production planning model

In a company, production planning and scheduling are located in different areas, i.e., at the strategic and operational levels, respectively. The goal of the production plan is to assign appropriate production amounts to resources in consideration of their capacity during the planning period in response to demands. On the other hand, in scheduling, the sequencing of jobs is decided to meet the given objective.

This study employed the production planning model presented by Kim and Lee (2016). Production planning is modeled as linear

Table 1

Notations for production planning model.

Notation	Definition
i	Wafer type ($i = 1, 2, \dots, I$)
j	Layer ($j = 1, 2, \dots, J$)
m	Machine ($m = 1, 2, \dots, M$)
t	Time (Day) ($t = 1, 2, \dots, T$)
D_{it}	Demand for wafer type i at time t
AWT_{mt}	Available working time of machine m at time t
ST	Setup time for changeover
PT_{ijm}	Processing time of layer j of wafer type i on machine m
M_{ij}	A set of dedicated machines for layer j of wafer type i
LT_{ij}	Lead-time for layer j of wafer type i
NS_{mt}	Number of setup changeovers on machine m at time t
K_{ij}	K_{ij} : Integer part of $\sum_{j=1}^J LT_{ij}$
F_{ij}	F_{ij} : Fraction part of $\sum_{j=1}^J LT_{ij}$
w_1	Weight of demand satisfaction
w_2	Weight of appropriate WIP achievement
X_{ijmt}	Production quantity of layer j of type i on resource m at time t
WIP_{ijt}	WIP level of layer j of type i at time t
τ_{it}	Lack of production quantity of type i at time t against demand
ϵ_{it}	Excess production quantity of i at time t against demand
χ_{ijt}	Shortage between actual and target WIP level of layer j of type i at time t
ψ_{ijt}	Surplus between actual and target WIP level of layer j of type i at time t

programming, and the main role of the model is to assign the production quantity of each layer of each type of wafer to resources to meet demand satisfaction and to reduce the lead-time. The WIP flow, resource capacity, setup change, available WIP level, and lead-time are regarded as the constraints. The description of the model is as follows and notations are shown in Table 1.

$$\text{Min} w_1 \sum_i \sum_t (\tau_{it} + \epsilon_{it}) + w_2 \sum_i \sum_t (\chi_{it} + \psi_{it}) \quad (1)$$

$$WIP_{ijt} = WIP_{ij(t-1)} - X_{ijm(t-1)} + X_{i(j-1)m(t-1)} \forall i, j, t, \text{ s.t. } m \in M_{ij} \quad (2)$$

$$\sum_{j \geq j} WIP_{ijt} + \chi_{ijt} - \psi_{ijt} = \sum_{i=1}^{t+K_{ij}-1} X_{ijm} + F_{ij} \times X_{ijm(t+F_{ij})} \forall i, j, t, \text{ s.t. } m \in M_{ij} \quad (3)$$

$$\sum_i \sum_j PT_{ijm} \times X_{ijmt} \leq AWT_{mt} - ST \times NS_{mt} \forall m, t \quad (4)$$

$$\sum_{i=1}^t X_{ijmt} + \tau_{it} - \epsilon_{it} = \sum_{i=1}^t D_{it} \forall i, t, \text{ s.t. } m \in M_{ij} \quad (5)$$

$$X_{ijmt} \leq P_{ijt} \times WIP_{ijt} \forall i, j, t, \text{ s.t. } m \in M_{ij} \quad (6)$$

$$X_{ijmt}, WIP_{ijt}, \tau_{it}, \epsilon_{it}, \chi_{ijt}, \psi_{ijt} \geq 0 \forall i, j, t, \text{ s.t. } m \in M_{ij} \quad (7)$$

Eq. (1) contains two terms that need to be minimized: demand satisfaction and appropriate WIP. The first term, demand satisfaction, minimizes the gap between the target demand and turnout, and the second term reduces the gap between the actual and target WIP levels. Eq. (2) is for WIP flow conservation, and Eq. (3) is the lead-time constraint. Eq. (4) calculates the available working time of the resource in time t . Eq. (5) calculates the gap between the target demand and cumulative output. Lastly, Eq. (6) ensures that the production quantity is less than or equal to the available WIP level. Eq. (7) is non-negativity variables. As the aforementioned production planning model has been published (Kim & Lee, 2016), further details are available in their paper.

4.2. Deep reinforcement learning based scheduling within production plan

Recently, among diverse methods for scheduling, deep RL has attracted considerable attention by researchers and practitioners and has been employed in diverse industrial fields with the view of scheduling optimization (Atallah et al., 2017; Hubbs et al., 2020; Lee & Lee, 2020; Lee et al., 2020).

Diverse algorithms have been developed for deep RL, and the deep Q-network (DQN) has been adopted by many researchers. The DQN was developed from Q-learning proposed by Watkins (1989). While, in Q-learning, a Q-value is computed with a pair of states and is updated by Q-table, the DQN proposed by Mnih et al. (2015), which is a combination of Q-learning and an NN, generates the approximate Q-value via the NN and updates the weight of the NN using the replay memory and target network. Samples are stored in the replay memory and some of them are randomly extracted for learning from the memory. This method solves the characteristic of coupling between learning data and provides stable learning to control overfitting. In addition, the target network is employed to calculate the target value with a fixed parameter for particular steps because the target value changes in every step if a target network is not used.

A few studies that focused on scheduling in semiconductor manufacturing using deep RL have been reported. In particular, efforts were made to minimize the completion time or maximize the throughput and so forth. Recently, Lang et al. (2020) employed the DQN for allocating jobs to machines and selecting the sequences of the operation. They compared their method with a greedy randomized adaptive search procedures (GRASP) algorithm and showed that the proposed method was better than the GRASP algorithm.

In this study, scheduling based on deep RL is suggested to achieve the plan generated from a production planning model. Once the production plan is generated, the scheduling is implemented to attain the given plan. As mentioned above, in the scheduling phase, as the manufacturing production environment is dynamic, the agent learns the environment and responds to the changes. Before describing the deep RL algorithm, the concepts of state, action, and reward are explained in the next section.

4.3. State, action, and reward formulation

The representation of state and action embraces the behavior of the agents in the manufacturing environment. The state is defined as the agents observe machines in the manufacturing environment. Let s_t be a set of states at time t and $\bar{M}_t$ be a set of idle machines at time t . The state denotes the current information of the idle machine $m \in \bar{M}_t$, which is expressed as S_{mt} . s_t can be expressed as follows:

$$s_t = \left\{ S_{1t}, \dots, S_{|\bar{M}_t|t} \right\} \quad (8)$$

As mentioned above, the composition of S_{mt} is based on information of the machine m , which is described as follows:

$$S_{mt} = \left\{ CS_{mt}, \frac{NS_{mt}}{T}, \left(\frac{Q_{1jmt}}{\sum_{i,j} Q_{ijmt}}, \dots, \frac{Q_{Ijmt}}{\sum_{i,j} Q_{ijmt}} \right), \left(\frac{atp_{1jmt}}{ptp_{1jmt}}, \dots, \frac{atp_{Ijmt}}{ptp_{Ijmt}} \right) \right\} \quad (9)$$

Here, CS_{mt} indicates the current setup type of machine m at time t , which shows the type of the previous wafer. NS_{mt} denote the number of setup changeovers before time t on machine m . Q_{ijmt} is the number of layers j of wafer type i waiting in the queue that can be processed on machine m at time t . Further, ptp_{ijm} is the sum of the target production of layer j of wafer i on machine m for time T and atp_{ijmt} is the cumulative

Table 2
Notations for deep RL.

Notation	Definition
S_t	Set of states at time t
S_{mt}	State of machine m at time t
CS_{mt}	Current setup type of machine m at time t
Q_{ijmt}	Quantity of layer j of wafer type i in queue waiting in front of machine m at time t
atp_{ijmt}	Cumulative actual production quantity of layer j of wafer type i on machine m at time t
ptp_{ijt}	Sum of target production quantity of layer j of wafer type i on machine m for time T
a_t	Set of actions at time t
A_{mt}	the set of candidate actions for machine m at time t
r_t	Set of rewards at time t
R_{mt}	Reward at time t of machine m
wa	Weight assigned to action a
$\bar{M}_t$	Set of idle machines at time t
U_t	Set of layers of wafer types waiting in queue at time t
Y	Replay memory
L	Minimum training point
B	Target Q -network update frequency
E	Episode
C	Q -network update frequency
θ	Parameters for Q -network
θ^-	Parameters for target Q -network

actual production quantity of layer j of wafer type i at time t . These four features enable the agent to determine the current setup type of the machine, the number of waiting wafers for layer j of wafer type i , and the ratio of the actual production quantity to the planned quantity. The fourth feature is the role of the degree of achievement for the production plan at the current time, in which the maximum number is 1. This enables the agent to establish whether the target production is achieved.

Let A_{mt} be denoted as the set of candidate actions, layers of wafers that wait to be processed on machine m at time t , for machine m at time t . The action is a layer of a wafer selected by the agent, $a \in A_{mt}$. The set of actions at time t , a_t , is expressed as follows:

$$a_t = \left\{ A_{1t}, \dots, A_{|\bar{M}_t|t} \right\} \quad (10)$$

Lastly, a reward is compensation for the action (the layer j of the wafer type i) taken by the agent. the reward is designed that more rewards are received when the action chosen by the agent for an idle machine is included in the production plan. Therefore, it is designed to select an action preferentially that is included in the production plan when the agent chooses an action next time.

$$r_t = \left\{ R_{1t}, \dots, R_{|\bar{M}_t|t} \right\} \quad (11)$$

$$R_{mt} = wa \times \frac{Q_{ijmt}}{p_{ijm}} - ST \quad (12)$$

$\frac{Q_{ijmt}}{p_{ijm}}$ represents the total amount of time taken to process the chosen action. The weight wa can be a positive value when the chosen action is included in the production plan, and otherwise, the weight can be a negative value, which is to prioritize the action included in the plan generated by the production planning model. Also, the setup time is considered in the calculation of the reward. Setup time ST becomes 0 when the selected wafer type i on machine m is the same as the previous wafer type. If it is not the same, setup time takes a value. Table 2 lists the notations used for deep RL.

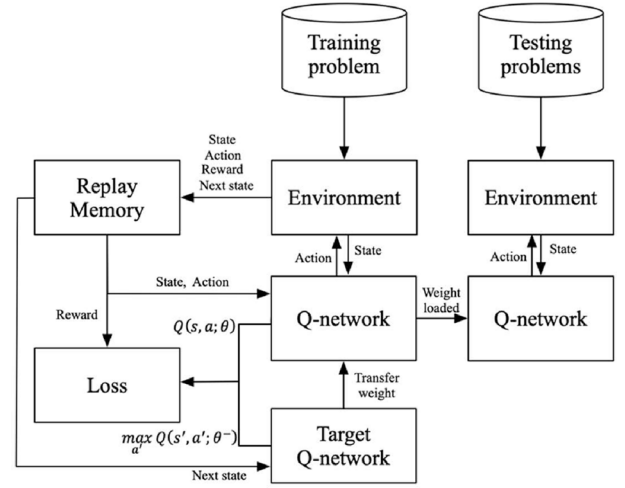


Fig. 2. Framework of DQN for training and testing.

4.4. Deep RL algorithm for scheduling within production plan

In this study, the DQN is the algorithm used for scheduling within the production plan, which is described in Algorithm 1. Let U_t be the set of layers of wafer types waiting in queues at time t . The layers of wafer types in U_t can be processed on idle resources, $\bar{M}_t$. When $|\bar{M}_t|$ and $|U_t|$ are both greater than zero, the wafer is scheduled by to the DQN algorithm until one of these sets is empty. During scheduling, the agent observes the state, which is the information of the idle machine, and executes the action that selects layer j of wafer i from U_t . The action is selected by employing the ϵ -greedy policy or by Q -value. A random number is generated, and if the number is less than or equal to the value of ϵ , the action is chosen randomly from the set of actions. If the number is larger than the value, then the action with the maximum Q -value is selected. The value of ϵ gradually decreases after the algorithm is trained such that the selection of actions using the maximum Q -value increasingly becomes necessary. To generate the Q -value, the state is combined with the additional information. This information is included with wafer type i , layer j , and processing time pt_{ijm} . Therefore, the final state combines the information about the machine with that of the candidate action to form the input for the NN network. The output is the Q -value, which is the value of the idle machine m when processing layer j of wafer i at time t . Algorithm 2 presents the pseudocode to show how an action is selected. Once an action is selected, the quantity of wafers of the selected layer j of the selected wafer type i in the queue is allocated to machine m .

After executing the action, the next state s_{t+1} is observed, and the agent receives reward r_t . The set of transitions (s_t, a_t, r_t, s_{t+1}) is stored in memory Y . The memory has the maximum length and if the memory is full, then the old transition is removed from the memory. After the number of sets of transitions occupies more than half of the size of the memory, training of the NN is commenced every C step. Lastly, the parameters of the target network are updated at every B episode after samples are saved over half of the size of the replay.

As mentioned, the proposed scheduling method is a basis for learning, thus training is performed in the first phase to learn the weights of the NN. In the training phase, the scheduling problem with the production plan is solved repeatedly by fixing the lead time of layer j of wafer i in the non-bottleneck process. Then, the trained weights of the NN is tested in the test phase with scheduling problems with production plans. At this time, the lead time in the non-bottleneck process is stochastic to examine the robustness of the proposed method, and dispatching rules. Fig. 2 shows the overall framework of the DQN for the training and testing phases.

Algorithm 1. DQN for scheduling with production planning

Input: Scheduling problem with production planning
Output: Weights θ of the Q -Network

- 1: Initialize replay memory Y
- 2: Initialize Q -Network with weights θ
- 3: Initialize target network Q with weights θ^-
- 4: X_{jmt} , WIP_{jt} are generated by optimizer for production planning
- 5: for $e = 1, E$ do
- 6: for $t = 1, T$ do
- 7: Get $\bar{M}_t$ and U_t
- 8: if $\bar{M}_t \neq \emptyset$ and $U_t \neq \emptyset$ then
- 9: repeat
- 10: Observe s_t
- 11: Select a random action a_t with probability ϵ greedy policy.
 Otherwise, $a_t = \max_a Q(s_t, a; \theta)$
- 12: Execute a_t (quantity of wafers of selected layer j of wafer i is assigned to $m \in \bar{M}_t$)
- 13: Observe s_{t+1} and r_t is given to the agent
- 14: Save transition (s_t, a_t, r_t, s_{t+1}) in Y
- 15: $\bar{M}_t \leftarrow \bar{M}_t \setminus \{m\}$, $U_t \leftarrow U_t \setminus \{i, j\}$
- 16: until $\bar{M}_t = \emptyset$ or $U_t = \emptyset$
- 17: end if
- 18: if $|Y| \geq L$ and Step %C = 0 then
- 19: Sampling random minibatch (s_k, a_k, r_k, s_{k+1}) from Y
- 20: Calculate loss
- 21: Perform a gradient decent regarding weights θ
- 22: $\epsilon \leftarrow \epsilon \times dr$
- 23: end if
- 24: Step $+= 1$
- 25: end for
- 26: Update $\theta^- = \theta$
- 27: end for

Algorithm 2. Action selection

- 1: rand $\leftarrow$ random ():
- 2: if rand $\leq \epsilon$ then
- 3: action $\leftarrow$ random (U_t)
- 4: else
- 5: action $\leftarrow$ max Q-value (S_{mt}, U_t)
- 6: end if

5. Computational experiments

In this section, the performance of the proposed scheduling method is compared by conducting computational experiments with several dispatching rules. As the semiconductor fabrication model developed by Kim and Lee (2016) was employed in this study, the settings of the manufacturing environment are the same as those in their study.

The manufacturing line consists of 60 machines and is designed to manufacture 10 wafer types and 20 layers. The machines are represented as photolithography bottlenecks. As mentioned earlier, the re-entry characteristic occurs because the wafers visit the machines 20 times. Ten machines are dedicated to each layer of each wafer type. The processing times for a wafer are uniformly generated in the range [1–2.6] min and differ in accordance with the layer of the wafer type and machine. Once a wafer type that differs from the previous one is processed on a machine, a setup time of 30 min is incurred for a changeover. The lead-time for each layer is distributed uniformly in the range [1.05–1.4] and the average number of days is 1.225.

This semiconductor fabrication model largely comprises two types of processes: non-bottleneck and bottleneck processes. As machines in the bottleneck process are considered, it is difficult to predict when wafer operations in non-bottlenecks are complete. Therefore, the lead-time for

non-bottleneck operations and moving time is assumed to be $\sigma_{ij} \cdot \mu + \exp((1 - \sigma_{ij}) \cdot \mu)$, where σ is the ratio of the processing time ($0 \leq \sigma_{ij} \leq 1$) and μ is defined as the average processing time. Thus, the moving time consists of the standard processing time and additional time. The standard processing time is assumed to be a constant value and can be calculated from historical data. Even though a wafer waits for operations in non-bottleneck processes, the expected additional time is the same as that of its starting times because of the memoryless property.

A total of 24 cases were used to evaluate the performance of the proposed scheduling method in comparison with the dispatching rules: three lead-time variations for non-bottleneck processes, four demand average cases, and two demand variation cases. The uncertainty in the lead-time for non-bottleneck processes is defined by σ_{ij} and a high value means that layer j of wafer type i has a high uncertainty. The values of σ_{ij} are distributed uniformly in the three cases: a range in [0.6–1.0] in case 1, in [0.5–1.0] in case 2, and in [0.4–1.0] in case 3. For demand average cases, the average daily demands for each wafer type are 350 in case 1, and (390, 390, 370, 370, 350, 350, 330, 330, 310, 310) in case 2, (400, 400, 375, 375, 350, 350, 325, 325, 300, 300) in case 3, and (450, 450, 400, 400, 350, 350, 300, 300, 250, 250) in case 4. The demand variations are 10 % and 20% for case 1 and case 2, respectively. The initial WIP level for each layer of each wafer is distributed uniformly in the range [0, average daily demand of wafer type/0.7] in each case.

Three dispatching rules were used to compare the performance: the PAB rule suggested by Kim and Lee (2016), the setup-based rule, and the target production-based rule. Dispatching rules select a layer of a wafer type among the layers of wafers, which can be processed on an idle machine. The PAB is the rule for the WIP balance that controls the level of the WIP. The advantage of this rule is to maintain the WIP balance properly so that responding to the demand variance is possible flexibly. On the other hand, it impacts negatively in the perspective of maximizing the throughput. This rule is mainly focused on minimizing the current WIP level and the target WIP level that is given by the production planning model.

In the setup-based rule, this rule pays attention to minimizing the loss time that occurred by the setup change. The setup change happens when the new job type is not the same as the previous one on the machine, which occurs the setup time on the machine to process the new job type. The setup time takes an important portion in the working day as the machine working time can be reduced due to the setup change-over, which can lead to decreasing the throughput. Therefore, the rule selects the same type of wafer among candidate wafers in order to decrease the time loss by the setup change. In the case of identical candidate wafer types, the quantity waiting in the queue is compared. For example, the current setup on the machine is wafer type 1, and 50 wafers for layer 3 of wafer type 1 and 30 wafers for layer 5 of wafer type 1 are waiting in the queue. In this case, layer 3 of wafer type 1 is selected to be processed on the machine. By selecting a layer of a wafer type for which a larger number of wafers are waiting in the queue, the opportunity for setup changeover is reduced. Therefore, the time loss for the setup change can be minimized and the throughput can be increased by allocating the wafer type that has a large number of wafers waiting in the queue first when identical candidate wafer types exist.

The plan-based rule prioritizes achieving the target quantity that is assigned to machines by the production planning. It can bring stability for satisfying the demand and fulfilling the production plan. However, it might have the weakness for time loss due to the setup changeover to accomplish the target quantity. For example, when one wafer for a certain wafer type is short for the target quantity, the wafer type is selected to fulfill the production plan. In this case, if the wafer type is different from the type on the machine, the time loss is huge comparing the production amount.

The plan-based rule considers target production quantity and current production quantity when selecting a layer of a wafer. When a machine is idle, the difference between the cumulative planned production

Table 3
Hyperparameters for deep RL.

Hyperparameter	Value
Size of replay memory (Y)	20,000
Optimizer	RMSprop
Batch size	64
Discount factor (γ)	0.9
Decaying rate (dr)	0.99
Minimum epsilon (ϵ)	0.1
Learning rate (lr)	0.001
Target Q -network update frequency (B)	20
Episode (E)	100
Q -network update frequency (C)	480
Minimum training point (L)	1000

quantity of layer j of wafer type i until the present time and the cumulative actual production quantity of layer j of wafer type i until the present time is calculated first. If the difference is positive, then the target planned production quantity of layer j of wafer type i was not achieved on the machine. Then, comparing the number of wafers for layer j of wafer type i waiting in the queue, the minimum value between the difference and the quantity is taken as the index. Layer j of wafer type i for which the index has the lowest value is selected. Thus, achievement of the planned production quantity is prioritized by selecting the layer of a wafer type with the smallest number of wafers waiting in the queue.

Four performance measurements were used, three of which were suggested by Kim and Lee (2016), and the last one was the throughput (TP) added in this study. The first performance measurement is feasibility, which is defined as the difference between the planned and actual production quantity. In other words, it indicates the practicality of the proposed scheduling method. The feasibility is calculated as follows:

$$Feasibility(FB) = average_{i,j,t} \left(\left| \sum_i X_{ijt}^p - \sum_i X_{ijt}^s \right| \right) \quad (13)$$

where X_{ijt}^p is the sum of the planned production quantity on all machines for layer j of wafer type i at time t , which is equal to $\sum_m X_{ijmt}$ and X_{ijt}^s is the actual production quantity for layer j of wafer type i at time t .

Other measurements are the demand satisfaction to determine the shortage of the output in the reward with the demand and lead time.

$$Demand\ satisfaction\ (DS) = average_{i,t} \left(\max \left(0, \sum_i D_{it} - \sum_i O_{it}^s \right) \right) \quad (14)$$

$$Leadtime(LT) = average_{i,j,t} \left(\frac{\sum_{v \in i} (ALT_{vj} \times P_{vjt})}{\sum_{v \in i} P_{vjt}} \right) \quad (15)$$

where O_{it}^s is the output of wafer type i at time t by the scheduling method, ALT_{vj} and is the actual lead time for layer j of wafer v belonging to wafer type i . P_{vjt} is the proportion of the time for which wafer v exists on layer j at time t .

The experiment was conducted on a computer with a 3.2 GHz Intel i7 processor with 16 GB of RAM, and the proposed algorithm was coded in Python. Also, the production planning model was solved by Gurobi 9.1.1, which is mathematical optimization solver. The NN that was used is fully connected and consists of four hidden layers: 64, 32, 16, and 8 layers. The activation function for all the layers was a leaky rectified linear unit. The hyperparameters that were used for the training and testing phases are provided in Table 3. As searching for the optimal hyperparameters is difficult because of the large search space, a random search (Bergstra & Bengio, 2012) was employed in this study to identify the best values.

In order to scrutinize the robustness of the presented method, in the experiment, the training of the NN was carried out in the manufacturing environment that the lead time in the non-bottleneck process was static, and the learned scheduling policy was tested with stochastic lead time in the non-bottleneck process.

To be specific, in the training phase, a production plan for 10 days was generated by the production plan model. The agents learned the scheduling policy in the semiconductor production environment with no uncertainty with respect to the lead time in non-bottleneck processes considering the 10 days generated plan while simulation ran for 100 episodes in Case 1 (1,1,1). During training, as mentioned above, if the random value is less than ϵ -value, the action is selected randomly. Otherwise, the action with the maximum value is chosen.

The curve of training during episodes was shown in Fig. 3 in the perspective of the throughput as the reward is meant to be maximizing the throughput. In the plot, the $\times$ and y indicate the episode and the total throughput that are produced for 10 days, respectively, and 34 times of training occurred for each episode approximately. As shown in Fig. 3, the convergence is presented up to 100 episodes as the throughput changes were negligible from the middle of the episodes. It was observed

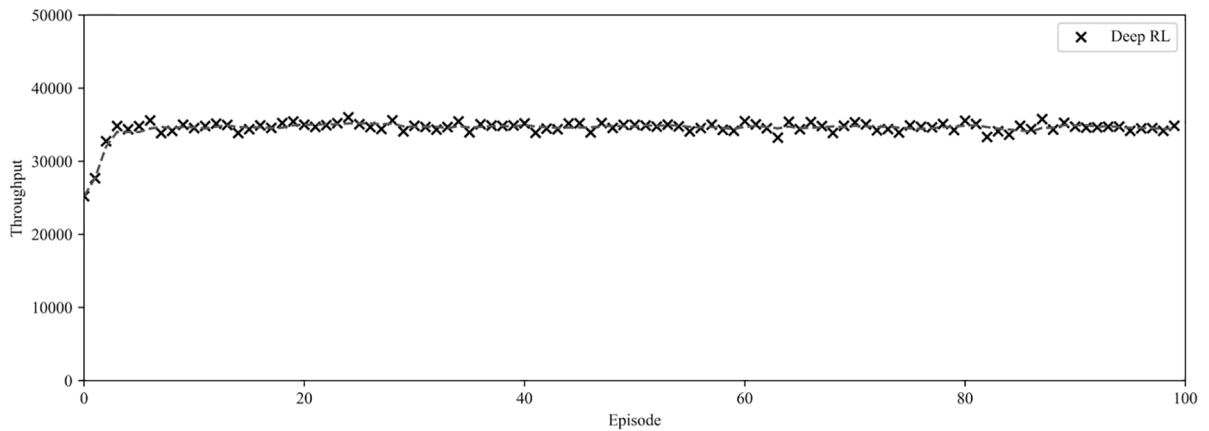


Fig. 3. Training results in perspective of throughput.

Table 4
Results for the proposed method in comparison with various dispatching rules.

Case	Throughput				Feasibility				Demand satisfaction				Lead time			
	Deep RL	Setup-based	PBB	Plan-based	Deep RL	Setup-based	PBB	Plan-based	Deep RL	Setup-based	PBB	Plan-based	Deep RL	Setup-based	PBB	Plan-based
1 (1,1,1)	47,011	37,036	38,452	37,395	289.3	329.3	289.3	288.3	130.7	200.6	162.7	171.1	2.255	2.627	2.979	2.990
2 (1,1,2)	55,168	41,471	41,058	41,310	295.5	329.9	301.6	300.1	88.1	179.6	164.8	157.7	2.135	2.523	3.357	3.274
3 (1,2,1)	52,492	40,434	39,638	39,518	293.0	340.5	307.0	305.7	103.7	199.5	169.1	169.7	2.299	3.005	3.179	3.298
4 (1,2,2)	47,977	39,010	40,350	40,206	291.1	320.8	287.9	286.0	106.9	180.2	153.5	154.9	1.961	2.548	2.968	2.991
5 (1,3,1)	47,503	43,229	38,249	38,027	282.4	329.5	287.0	285.5	132.1	179.9	165.8	168.9	2.116	2.536	3.015	3.013
6 (1,3,2)	53,501	43,370	41,964	42,500	299.4	323.3	294.4	293.6	93.4	185.5	158.2	148.1	2.215	3.197	3.211	3.078
7 (1,4,1)	57,487	37,957	41,022	42,650	285.2	331.3	298.3	298.8	89.3	198.9	173.7	158.9	2.153	2.375	3.363	3.463
8 (1,4,2)	50,270	43,329	37,406	37,806	293.4	311.5	290.8	292.0	120.8	174.9	176.2	171.9	1.850	3.422	3.019	2.950
Mean	51,426	40,729	39,767	39,926	291.2	327.0	294.6	293.7	108.1	187.4	165.5	162.7	2.123	2.779	3.136	3.132
9 (2,1,1)	54,814	43,033	39,539	39,991	276.6	318.7	285.4	288.9	99.1	161.5	159.6	157.2	2.200	3.044	3.920	3.971
10 (2,1,2)	52,783	44,251	39,017	38,821	291.0	309.1	282.7	284.2	106.4	163.4	151.5	148.6	2.032	2.878	3.483	3.491
11 (2,2,1)	51,057	44,655	43,318	43,193	288.7	327.9	277.1	277.6	107.1	162.9	143.2	150.6	2.018	3.468	3.646	3.609
12 (2,2,2)	54,082	42,525	44,013	44,709	292.5	316.9	290.0	290.2	98.6	167.8	146.5	143.5	2.169	3.256	3.830	3.905
13 (2,3,1)	52,886	38,210	41,747	41,128	290.1	309.0	284.9	286.5	93.6	178.0	142.9	150.8	2.133	2.899	3.749	3.794
14 (2,3,2)	52,886	45,045	38,708	38,555	290.1	276.0	274.7	275.6	93.6	131.0	154.8	155.0	2.133	2.136	3.486	3.419
15 (2,4,1)	49,843	39,586	39,077	39,056	286.9	324.4	287.0	287.2	115.7	186.4	160.8	160.6	1.933	2.887	3.608	3.632
16 (2,4,2)	56,549	46,373	41,847	42,166	279.2	316.2	289.0	289.1	91.0	158.6	148.0	152.7	2.202	3.505	3.885	3.763
Mean	53,112	42,960	40,908	40,952	286.9	312.3	283.9	284.9	100.6	163.7	150.9	152.4	2.102	3.009	3.701	3.698
17 (3,1,1)	47,228	36,803	36,232	36,619	280.7	296.2	275.2	276.6	123.8	187.4	165.6	159.8	2.008	2.953	3.370	3.399
18 (3,1,2)	47,179	39,760	36,726	36,179	288.4	320.0	278.6	276.2	112.5	163.0	148.2	150.7	2.102	2.682	3.172	3.239
19 (3,2,1)	48,861	42,245	41,942	41,380	272.3	309.7	280.0	280.0	126.8	167.0	148.5	149.2	1.950	2.876	3.626	3.502
20 (3,2,2)	48,216	40,015	39,253	39,741	282.2	319.7	281.7	278.4	128.1	179.2	158.5	150.5	2.167	2.906	3.366	3.306
21 (3,3,1)	46,394	36,975	36,590	36,329	280.2	322.5	280.5	281.8	119.5	181.2	159.5	159.9	2.194	3.216	3.387	3.498
22 (3,3,2)	50,986	45,152	41,409	41,648	293.0	314.7	280.7	278.3	100.2	155.3	147.7	149.3	1.848	2.743	3.486	3.389
23 (3,4,1)	52,243	43,542	42,936	44,522	278.2	311.0	281.2	281.4	107.2	173.2	149.4	143.1	1.874	3.159	3.618	3.603
24 (3,4,2)	44,942	34,397	35,048	34,269	287.2	317.4	285.3	283.7	136.3	190.9	166.0	168.4	2.088	2.757	3.159	3.190
Mean	48,256	39,861	38,767	38,836	282.8	313.9	280.4	279.5	119.3	174.6	155.4	153.9	2.029	2.912	3.398	3.391
Total Mean	50,931	41,183	39,814	39,905	286.9	317.7	286.3	286.1	109.3	175.2	157.3	156.3	2.084	2.900	3.412	3.407

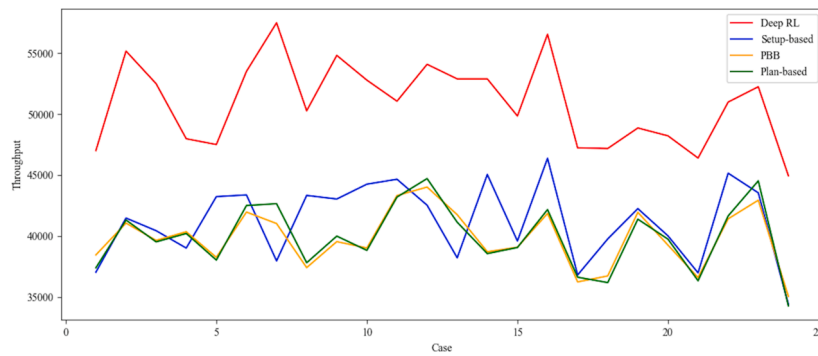


Fig. 4. Throughput trends for the proposed method and other rules.

that the throughput increased at the beginning of the episodes and did not change significantly when training proceeded continuously. It implies that the number of training can be decreased as the difference between their throughput of the episodes was not great, except for the beginning of the episodes. For the test, the weights of the NN that was saved at the last episode were employed.

As mentioned above, to test the robustness of the proposed method, the trained network was tested with 24 cases that reflected uncertainty in the lead-time. At this time, the simulation ran for 14 days with 14 days production plan. The performance of the proposed method was compared with three other dispatching rules for 24 cases. The results are described by considering the average values for 10 different instances in each case and are listed in Table 4.

The proposed method outperformed the dispatching rules in terms of throughput. The proposed method produced more 19% to 21% approximately, compared with the dispatching rules on the average throughput. Fig. 4 shows the throughput trend obtained by the proposed method and dispatching rules for all cases. The discrepancy between the throughput of the proposed method and that of the other rules was large in all cases. Comparing with the setup-based, the minimum quantity of the gap was 4274 and the maximum one was 19530. In the case comparing with the PBB rule, the minimum and the maximum value were 6919 and 16465. Lastly, 7481 and 14,837 were the minimum and the maximum difference between the proposed method and the plan-based rule.

In addition, the average lead-time of the proposed method decreased by approximately 39% to 63% compared with that of the dispatching rules. Even though the throughput by the proposed method was larger than other rules, the meaning of that the lead-time was shorter is that the proposed method showed better performance than the others. In terms of the feasibility and demand satisfaction measurements that reflect the accomplished degree for production plans, the proposed method fulfilled the production plan more effectively than the other rules. Both of these measurements are meant to execute the production plan; hence, values closer to 0 imply higher performance.

Moreover, the proposed method was able to more successfully produce the number of daily outputs in accordance with the production plan than the other rules, as confirmed by the demand satisfaction measurement. In particular, the proposed method was approximately 43% to 60% more efficient than the other rules. Regarding the feasibility, the result showed that the proposed method was better than setup-based rule but not the others. However, the gap of the value of the feasibility between the proposed method and the others was small. According to the results, in cases of the high uncertainty associated with

the non-bottleneck processes and the high variation in demand, the proposed method succeeded in improving the production efficiency of the manufacturing line compared with the other rules. In other words, it presented that the proposed method showed robust performance under dynamic environments.

Also, to observe the variations by hyperparameters, the sensitivity analysis of the hyperparameters was carried out. In this sensitivity analysis, some of the parameters were selected and set different values, to compare the parameters used in the above experiment. The details are shown in Appendix 1.

6. Conclusion

In Industry 4.0, advanced technologies enable communication between humans and machines for decision-making purposes, and the manufacturing paradigm has changed. Manufacturing systems become increasingly complicated as the number of products to be produced increases. Consequently, manufacturers are attempting to increase production efficiency by implementing advanced technologies to increase their competitiveness in the market and manage the complex manufacturing environment. Both researchers and practitioners have emphasized the need for a self-learning system to accomplish these objectives.

Production planning and scheduling are important processes in the manufacturing system, as they are directly related to production efficiency. In semiconductor fabrication, production planning and scheduling are highly critical because of the nature of semiconductor manufacturing such as the long production period and prevalence of re-entry processes. Scheduling is implemented in a dynamic manufacturing environment characterized by high uncertainty more than the production planning as it belongs to the environment that can happen unanticipated situations such as machine breakdown, jobs' insertion. Furthermore, if scheduling is unable to excel a good performance under the uncertainty of a dynamic environment, the best plan can be useless, which has a negative impact on the operation of the manufacturer.

In this paper, a scheduling method is proposed, which fulfills the production plan for semiconductor fabrication. The method is based on deep RL to provide robust solutions in a dynamic manufacturing environment. Deep RL is employed by using a novel state, action, and reward to express the manufacturing environment and the scheduling goal. Before testing the performance of the proposed method, an agent was used to train the NN in the manufacturing environment in a static situation in the training step. In the test step, the trained network was used to compare the performance of the proposed method with those of

dispatching rules and another deep RL method. In addition, by employing the centralized fashion of the NN, the variability of the number of machines and the number of layers of wafer type dedicated to machines were addressed. Therefore, new scheduling problems could be solved with a different number of machines by using the trained network. However, an increase in the number of layers or wafer type would require the NN to be retrained. The development of an NN that would not need to be retrained is a future task.

In addition, in the future, for further improvement, the practicality by including non-bottleneck processes in the model to circumvent having to express these processes as moving time could be contemplated. Furthermore, machine breakdown, deterioration, and so forth could be considered in the model for enhanced practicality in the manufacturing line.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

CRediT authorship contribution statement

Young Hoon Lee: Conceptualization, Methodology, Validation.
Seunghoon Lee: Conceptualization, Methodology, Validation, Software, Writing – original draft, Writing – review & editing.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Appendix 1. Sensitivity analysis of hyperparameters

To examine hyperparameters' sensitivity, some of the hyperparameters were selected; the NN architecture, replay memory size, and learning rate. The other hyperparameters remained the same as those in Table 3. Besides, in the experiment for the sensitivity, the procedure of the training and testing was the same as the above experiment.

The results are shown in Fig. A.1. In terms of the network architecture variance, the architecture (64, 32, 16, 8) showed the best performance in four performance measurements. A comparison of the architecture (512, 256, 128, 64) with the architecture (128, 64) revealed that the number of hidden layers did not significantly influence the results. In the case of the capacity of the replay memory, the result showed that, when the memory capacity is either too much or too little, this did not impact the learning positively. As the samples were randomly selected in the form of mini-batches from the replay memory for learning, the replay memory that contained the samples needs to be of the appropriate size. Likewise, in the case of the learning rate, a rate that is either too low or too high had a negative impact on learning, as the learning speed was related to finding the optimal weights.

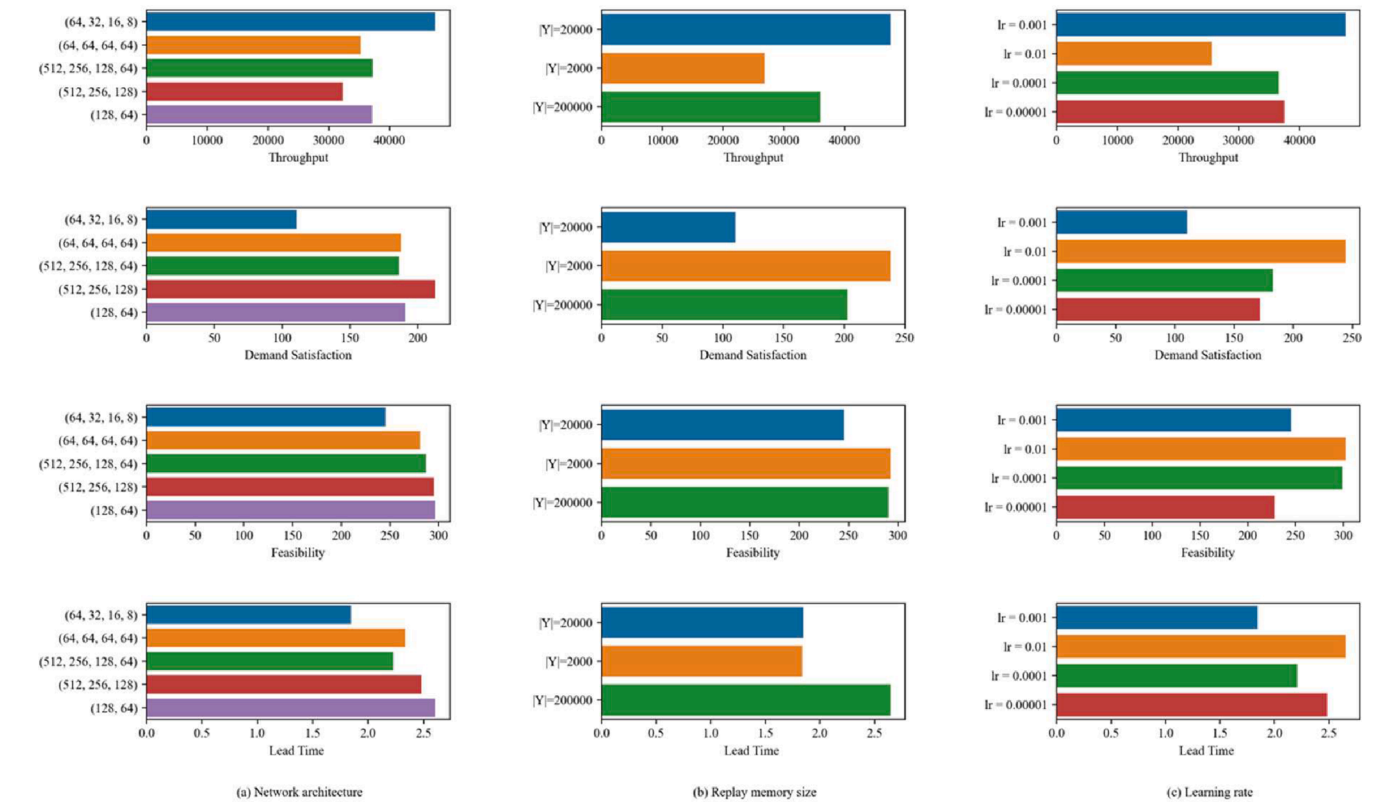


Fig. A.1. Sensitivity results for hyperparameters.

References

- Atallah, R., Assi, C., & Khabbaz, M. Deep reinforcement learning-based scheduling for roadside communication networks. In *2017 15th International Symposium on Modeling and Optimization in Mobile, Ad Hoc, and Wireless Networks (WiOpt)*, 2017 (pp. 1-8): IEEE.
- Bergstra, J., & Bengio, Y. (2012). Random search for hyper-parameter optimization. *The Journal of Machine Learning Research*, 13(1), 281–305.
- *Chen *, J. C., Chen, C.-W., Tai, C.-Y., & Tyan, J. C. (2004). Dynamic state-dependent dispatching for wafer fabrication. *International Journal of Production Research*, 42(21), 4547–4562.
- Connors, D., Feigin, G., & Yao, D. (1994). Scheduling semiconductor lines using a fluid network model. *IEEE Transactions on Robotics and Automation*, 10(2), 88–98.
- Dabbas, R. M., & Fowler, J. W. (2003). A new scheduling approach using combined dispatching criteria in wafer fabs. *IEEE Transactions on semiconductor manufacturing*, 16(3), 501–510.
- Demirkol, E., & Uzsoy, R. (1997). Performance of decomposition methods for complex workshops under multiple criteria. *Computers & Industrial Engineering*, 33(1-2), 261–264.
- Guo, C., Zhibin, J., Zhang, H., & Li, N.a. (2012). Decomposition-based classified ant colony optimization algorithm for scheduling semiconductor wafer fabrication system. *Computers & Industrial Engineering*, 62(1), 141–151.
- Hubbs, C. D., Li, C., Sahinidis, N. V., Grossmann, I. E., & Wassick, J. M. (2020). A deep reinforcement learning approach for chemical production scheduling. *Computers & Chemical Engineering*, 141, 106982. <https://doi.org/10.1016/j.compchemeng.2020.106982>
- Jain, V., Swarnkar, R., & Tiwari, M. K. (2003). Modelling and analysis of wafer fabrication scheduling via generalized stochastic Petri net and simulated annealing. *International Journal of Production Research*, 41(15), 3501–3527.
- Kaskavelis, Christos. A., & Caramanis, Michael. C. (1998). Efficient Lagrangian relaxation algorithms for industry size job-shop scheduling problems. *IIE Transactions*, 30(11), 1085–1097.
- Kim, S. H., & Lee, Y. H. (2016). Synchronized production planning and scheduling in semiconductor fabrication. *Computers & Industrial Engineering*, 96, 72–85.
- Kim, Y.-D., Lee, D.-H., Kim, J.-U., & Roh, H.-K. (1998). A simulation study on lot release control, mask scheduling, and batch scheduling in semiconductor wafer fabrication facilities. *Journal of Manufacturing Systems*, 17(2), 107–117.
- Kumar, S., & Kumar, P. (2001). Queueing network models in the design and analysis of semiconductor wafer fabs. *IEEE Transactions on Robotics and Automation*, 17(5), 548–561.
- Lang, S., Behrendt, F., Lanzerath, N., Reggelin, T., & Müller, M. Integration of Deep Reinforcement Learning and Discrete-Event Simulation for Real-Time Scheduling of a Flexible Job Shop Production. In *2020 Winter Simulation Conference (WSC)*, 2020 (pp. 3057-3068): IEEE.
- Lee, S., Cho, Y., & Lee, Y. H. (2020). Injection Mold Production Sustainable Scheduling Using Deep Reinforcement Learning. *Sustainability*, 12(20), 8718.
- Lee, Y. H., & Kim, J. W. (2011). Daily stepper scheduling rule in the semiconductor manufacturing for MTO products. *The International Journal of Advanced Manufacturing Technology*, 54(1-4), 323–336.
- Lee, S., & Lee, Y. H. (2020). Improving Emergency Department Efficiency by Patient Scheduling Using Deep Reinforcement Learning. *Healthcare*, 8(2), 77.
- Li, L. (2012). Multi-ant colony-based sequencing method for semiconductor wafer fabrication facilities with multi-bottleneck. *International Journal of Modelling, Identification and Control*, 15(4), 259–266.
- Liao, D.-Y., Chang, S.-C., Pei, K.-W., & Chang, C.-M. (1996). Daily scheduling for R&D semiconductor fabrication. *IEEE Transactions on semiconductor manufacturing*, 9(4), 550–561.
- Lu, S. C., Ramaswamy, D., & Kumar, P. (1994). Efficient scheduling policies to reduce mean and variance of cycle-time in semiconductor manufacturing plants. *IEEE Transactions on Semiconductor Manufacturing*, 7(3), 374–388.
- Luo, S. (2020). Dynamic scheduling for flexible job shop with new job insertions by deep reinforcement learning. *Applied Soft Computing*, 91, 106208. <https://doi.org/10.1016/j.asoc.2020.106208>
- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518(7540), 529–533.
- Park, I.-B., Huh, J., Kim, J., & Park, J. (2019). A Reinforcement Learning Approach to Robust Scheduling of Semiconductor Manufacturing Facilities. *IEEE Transactions on Automation Science and Engineering*.
- Pinedo, M. (2012). *Scheduling* (Vol. 29). Springer.
- Qiao, F., Ma, Y.-M., Li, L.i., & Yu, H.-X. (2013). A Petri net and extended genetic algorithm combined scheduling method for wafer fabrication. *IEEE Transactions on Automation Science and Engineering*, 10(1), 197–204.
- Shi, D., Fan, W., Xiao, Y., Lin, T., & Xing, C. (2020). Intelligent scheduling of discrete automated production line via deep reinforcement learning. *International Journal of Production Research*, 58(11), 3362–3380.
- Sourirajan, K., & Uzsoy, R. (2007). Hybrid decomposition heuristics for solving large-scale scheduling problems in semiconductor wafer fabrication. *Journal of Scheduling*, 10(1), 41–65.
- Stricker, N., Kuhnle, A., Sturm, R., & Friess, S. (2018). Reinforcement learning for adaptive order dispatching in the semiconductor industry. *CIRP Annals*, 67(1), 511–514.
- Sung, C. S., & Choung, Y. I. (2000). Minimizing makespan on a single burn-in oven in semiconductor manufacturing. *European Journal of Operational Research*, 120(3), 559–574.
- Tao, F., Qi, Q., Liu, A., & Kusiak, A. (2018). Data-driven smart manufacturing. *Journal of Manufacturing Systems*, 48, 157–169.
- Tao, F., Qi, Q., Wang, L., & Nee, A. (2019). Digital twins and cyber-physical systems toward smart manufacturing and industry 4.0: Correlation and comparison. *Engineering*, 5(4), 653–661.
- Tsai, C.-H., Feng, Y.-M., & Li, R.-K. (2003). A hybrid dispatching rules in wafer fabrication factories. *International Journal of the Computer, the Internet and Management*, 11(1), 64–72.
- Wang, J., Ma, Y., Zhang, L., Gao, R. X., & Wu, D. (2018). Deep learning for smart manufacturing: Methods and applications. *Journal of Manufacturing Systems*, 48, 144–156.
- Waschneck, B., Reichstaller, A., Belzner, L., Altenmüller, T., Bauernhansl, T., Knapp, A., et al. Deep reinforcement learning for semiconductor production scheduling. In *2018 29th annual SEMI advanced semiconductor manufacturing conference (ASMC)*, 2018a (pp. 301-306): IEEE.
- Waschneck, B., Reichstaller, A., Belzner, L., Altenmüller, T., Bauernhansl, T., Knapp, A., et al. (2018b). Optimization of global production scheduling with deep reinforcement learning. *Procedia CIRP*, 72, 1264–1269.
- Watkins, C. J. C. H. (1989). Learning from delayed rewards.
- Wein, L. M. (1988). Scheduling semiconductor wafer fabrication. *IEEE Transactions on Semiconductor Manufacturing*, 1(3), 115–130.
- Wen, H.-W., Fu, L.-C., & Huang, S.-S. Modeling, scheduling, and prediction in wafer fabrication systems using queueing Petri net and genetic algorithm. In *Proceedings 2001 ICRA. IEEE International Conference on Robotics and Automation (Cat. No. 01CH37164)*, 2001 (Vol. 4, pp. 3559-3564): IEEE.
- Zhang, J., Ding, G., Zou, Y., Qin, S., & Fu, J. (2019). Review of job shop scheduling research and its new perspectives under Industry 4.0. *Journal of Intelligent Manufacturing*, 30(4), 1809–1830.
- Zhong, R. Y., Xu, X., Klotz, E., & Newman, S. T. (2017). Intelligent manufacturing in the context of industry 4.0: A review. *Engineering*, 3(5), 616–630.